EWB CLOUD LABS

AWS VPC PEERING

Complete Student Project Submission — Report Template

Student Name: K. Sanjana

Batch / Section: [Enter batch/section]

AWS Region: [e.g., Asia Pacific (Mumbai) ap-south-1]

Completion Date: [DD Month YYYY]

Project: Connect two VPCs and prove private IPv4 connectivity

1. Abstract

This project demonstrates private communication between two Amazon Virtual Private Clouds (VPCs) using an AWS VPC peering connection. VPC A hosts a public bastion instance and a private application instance, while VPC B hosts a second private application instance. The two VPCs use non-overlapping IPv4 CIDR blocks so traffic can be routed unambiguously. A peering connection links the VPCs, and route tables on both sides provide forward and return paths. Security groups restrict SSH and ICMP traffic to specified lab hosts. Connectivity is validated through a baseline ping failure before peering routes are added, successful forward and reverse pings after configuration, SSH access to App-B through the bastion, and a controlled ICMP failure followed by recovery.

2. Objectives

• Create two VPCs with non-overlapping IPv4 CIDR blocks.

• Create public and private subnets and explicitly associate route tables.

• Launch one public bastion host and two private application instances.

• Establish and activate a VPC peering connection between VPC A and VPC B.

• Configure routes in both directions and restrict inbound traffic with security groups.

• Verify private connectivity, demonstrate a controlled failure, and clean up lab resources.

3. Architecture and IP Plan

VPC A (10.10.0.0/16) contains public subnet 10.10.1.0/24 with the bastion at 10.10.1.10, and private subnet 10.10.2.0/24 with App-A at 10.10.2.10. VPC B (10.20.0.0/16) contains private subnet 10.20.1.0/24 with App-B at 10.20.1.10. VPC A has an Internet Gateway for bastion administration. VPC B has no Internet Gateway. The peering connection carries private traffic between the VPCs; it does not provide VPC B with internet access.

Resource	CIDR / IP	Purpose
ewb-vpc-a	10.10.0.0/16	VPC A
ewb-a-public	10.10.1.0/24	Bastion subnet
ewb-a-private	10.10.2.0/24	App-A subnet
ewb-vpc-b	10.20.0.0/16	VPC B
ewb-b-private	10.20.1.0/24	App-B subnet
ewb-bastion	10.10.1.10 + public IP	Admin entry point
ewb-app-a	10.10.2.10	Private application host
ewb-app-b	10.20.1.10	Private application host

4. Resource Inventory — Fill Actual AWS IDs

Resource	Expected value	Actual ID / value
VPC A	ewb-vpc-a / 10.10.0.0/16	[vpc-…]
VPC B	ewb-vpc-b / 10.20.0.0/16	[vpc-…]
Public subnet A	ewb-a-public / 10.10.1.0/24	[subnet-…]
Private subnet A	ewb-a-private / 10.10.2.0/24	[subnet-…]
Private subnet B	ewb-b-private / 10.20.1.0/24	[subnet-…]
Peering	ewb-a-b-peer / Active	[pcx-…]
Bastion	10.10.1.10	[i-…; public IP: …]

Resource	Expected value	Actual ID / value
App-A	10.10.2.10; no public IP	[i-…]
App-B	10.20.1.10; no public IP	[i-…]
AWS Region	e.g. ap-south-1	[Your Region]

5. Implementation Procedure

Step 1 — Create VPC A and subnets

Create ewb-vpc-a with 10.10.0.0/16. Create ewb-a-public (10.10.1.0/24) and ewb-a-private (10.10.2.0/24).

Step 2 — Internet Gateway and routes

Attach ewb-igw-a to VPC A. Associate ewb-rt-a-public with the public subnet and add 0.0.0.0/0 → Internet Gateway. Associate ewb-rt-a-private with the private subnet and keep only the local route before peering.

Step 3 — Create VPC B

Create ewb-vpc-b with 10.20.0.0/16 and ewb-b-private with 10.20.1.0/24. Associate ewb-rt-b-private. Do not create an Internet Gateway for VPC B.

Step 4 — Configure security groups

Bastion: SSH TCP 22 from your current public IPv4 /32. App-A: SSH TCP 22 from 10.10.1.10/32 and ICMP Echo Request from 10.20.1.10/32. App-B: SSH TCP 22 from 10.10.1.10/32 and ICMP Echo Request from 10.10.2.10/32. Retain default outbound allow-all rules for this lab.

Step 5 — Launch EC2 instances

Launch Amazon Linux 2023 instances using the ewb-lab key pair. Bastion is in the public subnet with private IP 10.10.1.10 and a public IPv4 address. App-A and App-B use their specified private subnets and IPs, with public IPv4 disabled.

Step 6 — Establish peering

Create ewb-a-b-peer with VPC A as requester and VPC B as accepter in the same account and Region. Accept the request and wait for Active. Record the pcx ID.

Step 7 — Add routes

In ewb-rt-a-private add 10.20.0.0/16 → pcx-ID. In ewb-rt-b-private add 10.10.0.0/16 → pcx-ID. In ewb-rt-a-public add 10.20.0.0/16 → pcx-ID. Keep local routes and the public subnet's 0.0.0.0/0 → IGW route.

Step 8 — Configure SSH ProxyJump

Run SSH from your laptop. Configure the bastion public IPv4 and use ProxyJump for App-A and App-B. Keep the private key on your laptop.

6. Route Table Configuration

Route table / subnet	Destination	Target
ewb-rt-a-public / public A	10.10.0.0/16	local
ewb-rt-a-public / public A	0.0.0.0/0	ewb-igw-a
ewb-rt-a-public / public A	10.20.0.0/16	pcx-[actual ID]
ewb-rt-a-private / private A	10.10.0.0/16	local
ewb-rt-a-private / private A	10.20.0.0/16	pcx-[actual ID]
ewb-rt-b-private / private B	10.20.0.0/16	local
ewb-rt-b-private / private B	10.10.0.0/16	pcx-[actual ID]

7. SSH Configuration Example

Replace BASTION_PUBLIC_IP with the actual bastion public IPv4 address. Confirm the private IPs match the instances.

```
Host ewb-bastion
    HostName BASTION_PUBLIC_IP
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes

Host ewb-app-a
    HostName 10.10.2.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion

Host ewb-app-b
    HostName 10.20.1.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion
```

```
chmod 400 ~/.ssh/ewb-lab.pem
ssh ewb-bastion
ssh ewb-app-a
ssh ewb-app-b
```

8. Verification and Test Results

Paste your actual terminal output and screenshots. The expected results are acceptance criteria, not proof of completed execution.

Test	Command / expected result	Evidence / actual result
Baseline before peering routes	App-A: ping -c 4 10.20.1.10 — no replies	[Paste output]
Forward path	App-A ping App-B — 4 replies, 0% loss	[Paste output]
Reverse path	App-B ping App-A — 4 replies, 0% loss	[Paste output]
Private SSH	Laptop: ssh ewb-app-b — shell via bastion	[Paste screenshot]
Isolation	App-A and App-B have no public IPv4	[Paste screenshot]
Controlled failure	Remove App-B ICMP rule; ping should fail	[Paste output]
Recovery	Restore ICMP rule; ping should resume	[Paste output]

Test commands

```
hostname
hostname -I
ping -c 4 10.20.1.10    # on App-A
ping -c 4 10.10.2.10    # on App-B
```

9. Security Considerations

• Use non-overlapping VPC CIDRs.

• Keep application hosts private with no public IPv4 addresses.

• Restrict bastion SSH to your current public IP /32.

• Allow application SSH only from bastion private IP 10.10.1.10/32.

• Use specific /32 ICMP sources; never expose SSH or ICMP to 0.0.0.0/0.

• Never include private keys, passwords or tokens in the report.

• This beginner lab assumes default allow-all network ACLs.

10. Troubleshooting

Problem	Checks
Bastion SSH timeout	Check public IP, current client IP /32 rule, public subnet association and IGW route.
App-A SSH fails	Check private IP, TCP 22 source, key, ec2-user and instance state.
App-B SSH fails	Check public-subnet peering route, VPC B return route, App-B SSH rule and peering status.
Ping fails	Check both route tables, pcx ID, Active status, ICMP rule and actual host IPs.
Peering creation fails	Check for overlapping CIDRs, including secondary CIDRs.
Permission denied (publickey)	Check key, IdentityFile and ec2-user.

11. Viva Questions and Answers

1. What is VPC peering?

A private connection between two VPCs that permits private-IP communication when routes and security rules allow it.

2. Why must VPC CIDRs not overlap?

Overlapping ranges make destination routing ambiguous and prevent a valid peering design.

3. Why are return routes needed?

Replies need a valid path back to the original sender.

4. Why is App-B SSH source the bastion private IP?

The bastion initiates a separate SSH connection to App-B.

5. Does Active peering guarantee connectivity?

No. Routes and security rules must also permit traffic.

6. Can VPC B use VPC A’s Internet Gateway through peering?

No. Peering does not provide internet-gateway sharing or transitive routing.

7. What is a bastion host?

A controlled administration entry point to private instances.

8. Why use /32 security group sources?

A /32 restricts the rule to one IPv4 address.

9. Is peering transitive?

No. A-B and A-C do not automatically connect B-C.

10. What does the controlled failure prove?

Removing the ICMP rule blocks ping; restoring it should allow ping again.

12. Learning Outcomes

This project demonstrates CIDR planning, public/private subnet design, route-table association, Internet Gateway configuration, VPC peering, bidirectional routing, security group controls, bastion SSH access, connectivity testing and troubleshooting.

13. Cleanup

• Save evidence before deleting resources.

• Terminate all three lab instances and remove leftover lab volumes/snapshots.

• Release any Elastic IP and delete the lab key pair if no longer needed.

• Delete peer routes and the peering connection.

• Delete lab security groups after network interfaces disappear, then subnets, custom route tables and the detached Internet Gateway.

• Delete both VPCs and verify no lab resources remain in the Region; check Billing because reporting can lag.

14. Conclusion

This project connects two AWS VPCs privately through VPC peering while keeping application servers without public IPv4 addresses. A bastion host provides controlled administration. Successful connectivity depends on non-overlapping CIDRs, active peering, routes in both directions and appropriately scoped security groups. Baseline, positive, negative and recovery tests show how routing and security rules affect reachability. Resources should be cleaned up after evidence is saved to avoid unnecessary costs.

15. Final Submission Checklist

■ Cover page completed with student name, batch, Region and date.

■ Architecture and resource inventory updated with actual IDs/IPs.

■ All three route tables and subnet associations captured.

■ Active peering and security group rules captured.

■ Baseline failure and forward/reverse ping evidence included.

■ SSH to App-B through bastion shown.

■ Controlled ICMP failure and recovery shown.

■ Cleanup evidence and learning note included.

Suggested filename: K_Sanjana_EWB_VPC_Peering.pdf

Reference titles from the workbook: VPC peering concepts and limitations; Create a VPC peering connection; Update route tables for peering; Security groups for peered VPCs; EC2 key pair security; Amazon VPC pricing.